

GitHub CI/CD and Collaboration Documentation

Code Push/Pull Workflows, Branching, Pull Requests, Team Collaboration, and Basic GitHub Actions

Prepared for interview-ready understanding and practical project submission

Core idea: This report explains GitHub collaboration from basics to practical CI/CD workflow. It includes commands, examples, team workflow, GitHub Actions concepts, and interview-ready answers.

Section	Topics Covered
Git and GitHub	Version control, repository, local and remote workflow
Code Push/Pull	git status, add, commit, push, pull
Branching and PRs	Feature branches, merge, conflicts, pull requests
Team Collaboration	Professional workflow, commit messages, issues
CI/CD Basics	Continuous Integration, Delivery, Deployment, GitHub Actions
Best Practices	.gitignore, README, GitHub Secrets, security

1. Introduction

GitHub CI/CD and collaboration are important parts of modern software development. In a real project, developers do not simply write code and upload files manually. They use Git to track code changes, GitHub to collaborate online, branches to work safely, pull requests to review code, and CI/CD pipelines to automatically test and build the project.

This documentation explains the practical workflow used in individual and team projects. The goal is to understand how code moves from a local system to GitHub and finally to a stable production-ready version.

Important: In simple words, Git tracks code changes, GitHub stores the code online, and CI/CD automates testing, building, and deployment steps.

2. Git vs GitHub

2.1 What is Git?

Git is a version control system. It records the history of code changes and allows developers to manage different versions of a project. With Git, we can track who changed what, when the change was made, and why the change was made.

Git is useful because it helps us restore old versions, compare code changes, work on different features using branches, and collaborate with a team without losing code.

2.2 What is GitHub?

GitHub is a cloud-based platform used to host Git repositories online. It allows developers to store projects, collaborate with team members, create pull requests, review code, track issues, and automate CI/CD workflows using GitHub Actions.

Git	GitHub
Version control tool	Online platform for hosting Git repositories
Works locally on the system	Works on cloud/remote server
Tracks code history	Supports collaboration, pull requests, issues, and CI/CD
Command-line based workflow	Web interface plus Git integration

Interview line: Git is the tool. GitHub is the online platform where Git repositories are stored and shared.

3. Repository Basics

A repository, or repo, is a project folder that contains source code and Git history. A repository can be local, meaning it exists on the developer system, or remote, meaning it is hosted on GitHub.

Example project structure:

```
my-project/
├── frontend/
├── backend/
├── README.md
├── package.json
└── .gitignore
```

4. Basic Code Push Workflow

The basic Git workflow is used whenever we change code and want to save or upload it to GitHub.

```
git status
git add .
git commit -m "Add login page"
git push origin main
```

Command	Purpose
git status	Shows changed, staged, and untracked files
git add .	Stages all changed files for commit
git commit -m "message"	Saves a snapshot of staged changes locally
git push origin main	Uploads local commits to GitHub main branch

Interview line: git add stages changes, git commit saves changes locally, and git push uploads commits to the remote repository.

5. Pull Workflow

When working in a team, other members may push new changes to GitHub. Before starting work, it is a good habit to pull the latest changes from the remote repository.

```
git pull origin main
```

This command downloads and merges the latest changes from GitHub into the local branch. It helps avoid conflicts caused by working on outdated code.

Command	Simple Meaning
git push	Upload local commits to GitHub
git pull	Download latest changes from GitHub

6. Branching Strategy

6.1 What is a Branch?

A branch is a separate line of development. It allows developers to work on a new feature or bug fix without directly affecting the main branch. The main branch is usually kept stable.

```
git checkout -b feature-login
```

This command creates and switches to a new branch named feature-login.

6.2 Why Branches are Useful

- The main branch remains stable.
- Developers can work on different features at the same time.
- Bug fixes can be handled separately.
- Code review becomes easier through pull requests.
- Experimental work does not break production code.

6.3 Common Branching Strategies

Strategy	Description
Simple strategy	Use main branch and separate feature branches. Good

	for small projects.
Dev branch strategy	Use main for stable code, dev for testing/integration, and feature branches for tasks.
Feature branch workflow	Every new feature is developed in its own branch and merged through a pull request.

```

main
├── dev
│   ├── feature-login
│   ├── feature-dashboard
│   └── bugfix-navbar

```

Best practice: A common team strategy is feature branch -> dev -> main. The main branch should contain stable code.

7. Pull Requests

A Pull Request, or PR, is a request to merge changes from one branch into another branch. It allows team members to review the code before merging it into the main or dev branch.

feature-login ---> main

Pull Request: Please review and merge my login feature.

7.1 Why Pull Requests are Important

- They allow code review before merge.
- They help discuss changes with teammates.
- They allow CI checks to run before code becomes part of the main branch.
- They reduce bugs and improve code quality.
- They provide a clear history of why a change was made.

Important: A pull request is not just a merge button. It is a collaboration and review process.

8. Merge and Merge Conflicts

Merge means combining changes from one branch into another. For example, after a feature branch is reviewed, it can be merged into main.

feature-login + main = updated main branch

8.1 Merge Conflict

A merge conflict occurs when Git cannot automatically combine changes. This usually happens when two developers change the same line or same part of a file differently.

Conflict resolution: A merge conflict is solved manually by choosing the correct code, removing conflict markers, testing the result, and committing the resolved file.

9. Team Collaboration Workflow

In professional team projects, developers usually avoid pushing directly to the main branch. They use a structured workflow to keep the code stable.

1. Pull latest code from main/dev
2. Create a new feature branch
3. Make code changes
4. Commit with a clear message
5. Push feature branch to GitHub
6. Create a Pull Request
7. Run CI checks
8. Review and approve
9. Merge into dev or main

```
git pull origin main
git checkout -b feature-auth
# make code changes
git add .
git commit -m "Add authentication API"
git push origin feature-auth
```

10. Good Commit Messages

A commit message should clearly describe what change was made. Good commit messages help teammates understand the project history.

Bad Commit Message	Better Commit Message
update	Update README with setup steps
fix	Fix login form validation
final changes	Add API integration documentation
code	Add user CRUD API routes

Rule: A good commit message is short, clear, and action-based.

11. .gitignore

.gitignore is a file used to tell Git which files or folders should not be tracked or pushed to GitHub. It is important for avoiding unnecessary files and protecting sensitive information.

```
# Example .gitignore
.env
node_modules/
venv/
__pycache__/
dist/
build/
*.log
```

Files like .env should not be pushed because they may contain secret keys, database URLs, or API tokens. Folders like node_modules and venv are generated locally and should not be stored in GitHub.

12. README.md

README.md is the introduction page of a GitHub repository. It helps users, teachers, recruiters, or teammates understand the project quickly.

README Section	Purpose
Project title	Name of the project
Description	What the project does
Tech stack	Technologies used
Features	Main features of the project
Setup steps	How to install and run the project
Screenshots	Visual proof of the project UI

```
# Project Name
```

```
## Description  
Short description of the project.
```

```
## Tech Stack  
- Python  
- FastAPI  
- React
```

```
## How to Run  
``bash  
pip install -r requirements.txt  
uvicorn main:app --reload  
``
```

13. GitHub Issues

GitHub Issues are used to track bugs, tasks, improvements, and feature requests. In a team, issues help organize work and assign tasks to members.

Issue Example	Meaning
Fix login validation bug	Bug tracking
Add user profile page	Feature request
Update documentation PDF	Task tracking
Resolve CORS error	Technical issue

14. GitHub Secrets

GitHub Secrets are used to securely store sensitive values required by CI/CD workflows, such as API keys, cloud credentials, database URLs, and tokens.

Examples of GitHub Secrets:

```
DATABASE_URL  
JWT_SECRET  
AWS_ACCESS_KEY_ID  
AWS_SECRET_ACCESS_KEY  
API_KEY
```

Security: Secrets should never be hardcoded in source code or committed to GitHub. They should be stored securely in GitHub Secrets or environment variables.

15. Basic CI/CD Concepts

15.1 What is CI?

CI stands for Continuous Integration. It means automatically checking, building, and testing code whenever developers push changes or create pull requests. CI helps catch bugs early before code is merged.

15.2 What is CD?

CD means Continuous Delivery or Continuous Deployment. Continuous Delivery prepares the code for release after successful checks. Continuous Deployment goes one step further and automatically deploys the code to production after checks pass.

Term	Meaning
Continuous Integration	Automatically test/build code after changes
Continuous Delivery	Prepare code for deployment after checks
Continuous Deployment	Automatically deploy code after checks pass

Simple meaning: CI checks code quality. CD helps deliver or deploy the application faster and safely.

16. GitHub Actions

GitHub Actions is GitHub's automation tool for CI/CD. It can automatically run workflows when code is pushed or when a pull request is created.

Workflow files are stored inside the `.github/workflows` folder.

```
.github/  
└─ workflows/  
    └─ ci.yml
```

GitHub Actions Term	Meaning
Workflow	Automation file that defines CI/CD process
Event	Trigger such as push or pull_request
Job	Group of steps running on a machine
Runner	Machine where workflow runs, such as ubuntu-latest
Step	Individual command or action inside a job

17. Example GitHub Actions Workflow - Node.js

```
name: Node.js CI
```

```
on:
```

```
  push:
```

```
    branches: [main]
```

```
  pull_request:
```

```
    branches: [main]
```

```
jobs:
  build:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version: 20

      - name: Install dependencies
        run: npm install

      - name: Run build
        run: npm run build
```

This workflow runs whenever code is pushed to main or a pull request is created for main. It checks out the code, sets up Node.js, installs dependencies, and runs the build command.

18. Example GitHub Actions Workflow - Python/FastAPI

```
name: Python CI

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.11"

      - name: Install dependencies
        run: |
          pip install -r requirements.txt

      - name: Run tests
        run: |
          pytest
```


For a Python project, GitHub Actions can install dependencies and run tests automatically. If the project does not have tests yet, this workflow can be modified to only install dependencies or run basic checks.

19. CI/CD Flow in a Real Project

Developer creates feature branch
↓
Developer pushes code
↓
Pull Request is created
↓
GitHub Actions runs CI checks
↓
Team reviews code
↓
PR is approved
↓
Code is merged into main/dev
↓
CD workflow deploys app to server/cloud

This workflow improves code quality because every change is reviewed and automatically checked before it becomes part of the stable branch.

20. Best Practices

- Do not push directly to main in team projects.
- Create a separate branch for every feature or bug fix.
- Write clear commit messages.
- Pull latest changes before starting work.
- Use pull requests for review.
- Do not commit .env files, secrets, or API keys.
- Use .gitignore for unnecessary files.
- Keep README.md updated.
- Use GitHub Issues to track tasks and bugs.
- Use GitHub Secrets for CI/CD credentials.

21. Interview-Ready Questions and Answers

What is Git?

Git is a version control system used to track code changes and manage project history.

What is GitHub?

GitHub is a cloud-based platform used to host Git repositories and collaborate with teams.

What is the difference between Git and GitHub?

Git is a local version control tool, while GitHub is an online platform for hosting and collaborating on Git repositories.

What is a repository?

A repository is a project folder that contains code and Git history.

What is git push?

git push uploads local commits to the remote repository on GitHub.

What is git pull?

git pull downloads and merges the latest changes from the remote repository into the local branch.

What is a branch?

A branch is a separate line of development used to work on features or fixes without affecting the main branch.

What is a Pull Request?

A Pull Request is a request to merge changes from one branch into another after review and checks.

What is a merge conflict?

A merge conflict occurs when Git cannot automatically combine changes because the same part of a file was modified differently.

What is CI/CD?

CI/CD is a practice that automates testing, building, and deployment of code. CI means Continuous Integration, and CD means Continuous Delivery or Deployment.

What is GitHub Actions?

GitHub Actions is GitHub's automation tool used to create CI/CD workflows for testing, building, and deploying code.

What is .gitignore?

.gitignore tells Git which files or folders should not be tracked or pushed to GitHub.

What are GitHub Secrets?

GitHub Secrets securely store sensitive values like API keys, database URLs, and deployment credentials for workflows.

22. Final Summary

GitHub collaboration and CI/CD are essential for modern software projects. Git helps track code changes, GitHub helps store and collaborate on code, branches allow safe feature development, pull requests support code review, and GitHub Actions automates testing, building, and deployment. A good team workflow keeps the main branch stable, uses feature branches, reviews code through pull requests, protects sensitive data using .gitignore and GitHub Secrets, and uses CI/CD checks before merging or deploying code.

Final interview line: In a professional workflow, developers create feature branches, commit changes, push them to GitHub, open pull requests, run CI checks using GitHub Actions, and merge only reviewed and stable code into the main branch.